CENTRO DE ESPECIALIZACIÓN EJECUTIVA - FIIS UNI

INFORME TÉCNICO DE VIABILIDAD TECNOLÓGICA: Tutor Virtual 24/7

Grupo	Grupo 5
Documento base	Avance_Investigacion_Tecnologica_Grupo5_Tutor_Virtual.pdf

RESTRICCIÓN NO NEGOCIABLE: COSTO TOTAL INCREMENTAL S/ 0

El stack definitivo no puede requerir licencias, suscripciones, APIs de pago, hosting comercial, dominios nuevos, almacenamiento contratado ni compra de hardware. La viabilidad productiva supone reutilizar infraestructura, conectividad, subdominio/certificado y almacenamiento ya disponibles en la UNI/CEE. Si esos recursos institucionales no existen, el despliegue productivo no es viable bajo S/ 0 y debe reducirse el alcance, no sustituirse por un servicio de pago.

1. Resumen Ejecutivo y Diagnóstico

El Tutor Virtual puede mantener costo monetario incremental S/ 0 únicamente si existe hardware institucional suficiente para ejecutar inferencia local. El documento original ya reconoce correctamente que “software gratuito” no equivale a infraestructura gratuita y que 24/7 depende de energía, red y equipo. Bajo la nueva restricción, se elimina cualquier dependencia de APIs comerciales, Ollama Cloud, Chroma Cloud, hosting, túneles o dominios pagados. La arquitectura recomendada conserva RAG local, FastAPI y un LLM local, pero consolida datos y vectores en PostgreSQL + pgvector para reducir servicios. La elección del modelo no debe fijarse antes del benchmark de CPU/RAM/GPU/VRAM; si el equipo existente no alcanza latencia y concurrencia mínimas, el alcance deberá reducirse en lugar de comprar hardware o consumir una API de pago.

2. Auditoría del Stack Propuesto

Tecnología original	Diagnóstico	Corrección costo S/ 0	Riesgo
Ollama local	Muy conveniente para prototipo e inferencia local sin pago por tokens.	Mantener, detrás de una interfaz de inferencia; prohibir uso de cloud en el proyecto.	MEDIO
LLM por definir	Decisión correcta; depende totalmente del hardware y calidad en español.	Seleccionar solo modelos cuya licencia permita uso institucional sin costo y mediante benchmark reproducible.	ALTO
FastAPI	Adecuado para RAG, streaming y conversaciones.	Mantener.	BAJO
React + Vite	Adecuado para chat y streaming.	Mantener con TypeScript; compilar y servir localmente.	BAJO
SQLite	Útil en demo, menos adecuado para concurrencia, auditoría y crecimiento.	Sustituir por PostgreSQL autoalojado.	ALTO
ChromaDB	Funcional, pero añade otro servicio y backup.	Consolidar vectores en PostgreSQL + pgvector para reducir operación.	MEDIO
Sentence Transformers	Adecuado para embeddings locales.	Mantener solo con modelo multilingüe/permisivo validado con preguntas CEE.	MEDIO
Cloudflare Tunnel	Puede ser gratuito, pero introduce dependencia externa y privacidad.	Eliminar del núcleo. Acceso por red interna o subdominio institucional existente.	MEDIO
Docker Engine/Compose + Nginx	Correctos para servidor único.	Mantener autoalojados.	BAJO

3. Arquitectura y Tecnologías Definitivas (El Stack Ideal)

Frontend / Interfaz: **React + TypeScript + Vite.** Interfaz de chat con streaming, historial, fuentes y estados de "información no encontrada", servida desde infraestructura institucional.

Backend / Procesamiento: **Python soportado + FastAPI + Pydantic + RAG propio.** Mantiene el flujo pequeño, auditable y sin frameworks de agentes innecesarios ni APIs externas.

Base de Datos / Almacenamiento: **PostgreSQL 18.x + pgvector + almacenamiento local de documentos.** Unifica usuarios, chats, documentos, versiones, chunks, metadatos y embeddings en un único motor libre.

Infraestructura, Despliegue y DevOps: **Ubuntu Server 24.04 LTS + Docker Engine/Compose + Nginx + systemd + Git.** Servidor/PC/VM institucional existente; acceso externo solo mediante red/subdominio ya provisto por UNI/CEE. TLS con certificado institucional o Let's Encrypt.

Herramientas Auxiliares: **Ollama local + modelo LLM open-weight/permisivo + modelo de embeddings local.** Cero pago por consulta. Se prohíbe cualquier fallback automático a servicios cloud de IA.

Condición de viabilidad del "24/7"

S/ 0 no puede crear capacidad física inexistente. Si el CEE/UNI no dispone de equipo, red y energía suficientemente disponibles, la salida correcta es un MVP interno/de horario limitado o menor concurrencia; no contratar cloud ni comprar GPU bajo esta restricción.

4. Matriz de Sostenibilidad y Riesgos

Componente	Curva	Costos de infraestructura	Obsolescencia	Seguridad	Mitigación
FastAPI + Python	Media	S/ 0	Bajo	Alta	Validación, rate limiting, streaming y parches.
PostgreSQL 18 + pgvector	Media	S/ 0 si existe servidor	Muy bajo	Muy alta	Backups, índices y roles.
Ollama local	Baja-Media	S/ 0 monetario; consume CPU/GPU existente	Medio	Alta	No exponer puerto; fijar versión y modelo.
LLM local	Alta	S/ 0 solo con hardware existente	Alto por evolución de modelos	Media-Alta	Benchmark, cuantización, límite de contexto y concurrencia.
Embeddings locales	Media	S/ 0	Medio	Alta	Medir retrieval en español; versionar embeddings.
Nginx/Docker/systemd	Media	S/ 0	Muy bajo	Alta	Red interna, TLS, restart y logs.
Infraestructura física	Alta	S/ 0 solo si ya existe	N/A	Depende de TI	Si el hardware no alcanza, reducir alcance. No existe alternativa externa garantizada a costo cero y 24/7.

5. Recomendaciones de Ingeniería de Procesos

Patrón recomendado: Monolito Modular + RAG local. La prioridad de ingeniería es medir calidad de recuperación, latencia y capacidad del equipo real; no crear microservicios.

1. Realizar Fase 0: inventario de CPU, RAM, GPU/VRAM, disco, red, disponibilidad eléctrica y usuarios simultáneos. Esta fase determina el alcance real S/ 0.
2. Crear un conjunto de preguntas reales del CEE y respuestas esperadas; medir retrieval y calidad antes de elegir LLM/embedding.
3. Implementar rechazo por baja relevancia: si no hay contexto suficiente, indicar que no se encontró información y derivar al canal humano.
4. Versionar documentos por curso, fecha y estado; al reemplazar material, retirar el contenido viejo del índice.
5. Aislar instrucciones del sistema, texto recuperado y pregunta del usuario para reducir prompt injection; el LLM no recibe permisos para ejecutar acciones críticas.

6. Medir primer token, latencia total, uso de RAM/VRAM y concurrencia; limitar usuarios si el hardware institucional lo exige.
7. Respaldar PostgreSQL y documentos en almacenamiento institucional existente y probar restauración.
8. No almacenar datos personales o conversaciones más tiempo del necesario; definir retención y acceso con TI/CEE.

Fuentes técnicas verificadas

Consulta de vigencia: 22/09/2026. Se priorizaron fuentes oficiales y documentación de los proyectos.

- Ollama - API/licencia: fuente oficial
- pgvector: fuente oficial
- PostgreSQL - política de versionado: fuente oficial
- Docker Engine: fuente oficial
- Ubuntu 24.04 LTS: fuente oficial